

Take-Home Assignment for Segment 08

Write a program to test statistical significance of Pearson product-moment correlation coefficient (https://en.wikipedia.org/wiki/Pearson_correlation_coefficient) by permutation test, using random sampling from all possible permutations.

Input:

1. A tab-delimited text file with two columns of numbers (and possibly additional columns that you should ignore; see the sample input files on eLC – these are GC content and GTG start codon percentages compiled from your earlier assignment).
2. The number of permutations to be sampled.

Output: The value of the correlation coefficient and the two-tailed p-value

C++ has a library function to generate permutations. However, I want you to write your own code to for the permutations and I will deduct 5 points if you use a C++ library function that does it for you.

Maximum reward for completing the assignment is 15 points.

Extra Credit Option:

If the number of permutations to be sampled (input item 2) is larger than the number of all possible different permutations ($N!$, where N is the number of paired values (rows) in the input file), use instead all possible permutations (if not, use the random sampling as described above). Write the program such that it decides itself which method to use.

Maximum reward for adding this to your code is 5 points.

More Extra Credit Opportunities:

We'll have a competition for the fastest implementation of the program. To time the program, write "time" at the beginning of the line. For example:

```
time ./CorCoefTest SampleInput.txt 10000000
```

To take part in the competition, your program must compile and run in the Linux shell, it has to work correctly, use mt19937 random number generator (not rand or any other faster generator), and it has to perform 10,000,000 permutations in a reasonable time (a couple of minutes max). You do not need to implement the extra credit option to participate in the competition.

I expect you to use the -O option (turning on code optimization) when you compile the program; you do not have to disadvantage yourself by not using it.

Prizes: 5, 3, 2, and 1 extra credit points for the 1st, 2nd, 3rd, and 4th place, respectively, plus 2 points and bragging rights for anyone whose program is faster than mine (or close enough to call it a tie)

Notes and recommendations:

- You can use some code we wrote earlier in the class. You can also find it on eLC (programs 083, 081, 040 are the most relevant to this assignment).
- Leave one array (let's say X) in the original order and permute the second array (Y) to simulate the conditions of the null hypothesis.
- Use the sample formula for the correlation coefficient:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

- To generate random permutations of an array, use the Fisher-Yates algorithm (https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle; also used in the code on eLC)
- For the extra credit part, you need to generate all possible permutations of the order of elements in the array without using any twice. You can use the Heap's algorithm (https://en.wikipedia.org/wiki/Heap%27s_algorithm), which does exactly that. Note that implementing the Heap's algorithm is much more difficult than Fisher-Yates shuffle.
- Sample input files that you can use are attached to the assignment on eLC.
- Sample output is also on eLC. Note that due to random sampling, your result may be slightly different than mine (when using all permutations, it should be exactly the same).
- If there is something you do not understand or are curious about, don't be afraid to ask. This assignment requires some knowledge of statistics and if you do not have that, I can help you. I will also review the required statistics background in the class and talk about code optimization.

Submit your code via eLC.